

République Tunisienne  
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique  
Université / École d'Ingénieurs — [Nom de l'établissement]

## **RAPPORT DE PROJET DE FIN D'ÉTUDES**

En vue de l'obtention du Diplôme National d'Ingénieur  
*Spécialité : Sécurité des Systèmes Informatiques et des Réseaux*

# **Plateforme de tests d'intrusion autonome pilotée par IA agentique : intégration du moteur Strix, d'un service RAG et de Metasploit, et conception d'une console web d'opérations de sécurité (Strix Console)**

Réalisé par : [Nom et Prénom de l'étudiant]

Encadrant académique : [Nom]

Encadrant professionnel : [Nom]

**Année Universitaire 2025 – 2026**

## Dédicace

À mes chers parents, pour leur amour inconditionnel, leurs sacrifices et leur confiance indéfectible qui ont été ma plus grande source de force tout au long de ce parcours.

À ma famille et à mes proches, pour leur soutien constant et leurs encouragements.

À mes enseignants et encadrants, pour la qualité de leur accompagnement et la rigueur qu'ils m'ont transmise.

À mes amis et camarades, pour tous les moments partagés qui ont donné du sens à cette expérience.

## Remerciements

Au terme de ce travail, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce projet de fin d'études.

Je remercie chaleureusement mon encadrant académique pour sa disponibilité, ses conseils avisés et son suivi rigoureux, ainsi que mon encadrant professionnel pour la confiance qu'il m'a accordée et pour le cadre technique stimulant qu'il a su instaurer.

Mes remerciements s'adressent également aux membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer ce travail, ainsi qu'à l'ensemble du corps enseignant pour la formation de qualité dont j'ai bénéficié.

## Résumé

Ce projet de fin d'études porte sur la conception et la réalisation d'une plateforme complète de tests d'intrusion autonomes, pilotée par un système d'intelligence artificielle agentique fondé sur des grands modèles de langage (LLM). Le travail articule quatre briques complémentaires en un système cohérent. (1) Le moteur agentique open-source Strix orchestre des agents qui exécutent des outils offensifs réels dans un bac à sable Docker/Kali. (2) Un service de connaissances RAG (Retrieval-Augmented Generation) auto-hébergé — FastAPI, base vectorielle Qdrant, embeddings et reranking neuronal — ancre le raisonnement des agents dans un corpus offensif vérifié afin de réduire les hallucinations. (3) Le framework d'exploitation Metasploit apporte un arsenal éprouvé ; un outil unique et déterministe, `exploit_research`, collapse le protocole RAG + Metasploit en un seul appel et produit une recommandation « module d'abord », déplaçant la décision d'exploitation du modèle vers du code vérifiable. (4) Une console web professionnelle, Strix Console (Next.js + FastAPI), transforme cet outillage en un véritable service d'opérations de sécurité : authentification forte (cookies `httpOnly`, CSRF, MFA TOTP), contrôle d'accès par rôles (RBAC), pilotage et suivi temps réel des campagnes (WebSocket), visualisation de la sortie du moteur, post-exploitation, triage des vulnérabilités, comparaison de scans, planification récurrente, export de rapports et journal d'audit inaltérable. La plateforme a été validée par 39 tests automatisés et vérifiée de bout en bout.

### Mots-clés :

*tests d'intrusion, IA agentique, LLM, Strix, RAG, Qdrant, Metasploit, RBAC, MFA, FastAPI, Next.js, sécurité applicative.*

## Abstract

This graduation project designs and builds a complete platform for autonomous penetration testing, driven by an agentic AI system based on Large Language Models (LLMs). It integrates four complementary building blocks into a coherent system. (1) The open-source Strix agentic engine orchestrates agents that run real offensive tools inside a Docker/Kali sandbox. (2) A self-hosted Retrieval-Augmented Generation (RAG) knowledge service — FastAPI, Qdrant vector store, neural embeddings and reranking — grounds the agents' reasoning in a verified offensive corpus to reduce hallucinations. (3) The Metasploit framework provides a battle-tested arsenal; a single deterministic tool, `exploit_research`, collapses the RAG + Metasploit protocol into one call and returns a module-first recommendation, moving exploitation decisions from the model to verifiable code. (4) A professional web console, Strix Console (Next.js + FastAPI), turns this tooling into a real security-operations service: strong authentication, RBAC, real-time orchestration (WebSocket), engine-output visualisation, post-exploitation, findings triage, scan diffing, recurring scheduling, report export and a tamper-evident audit trail. The platform is covered by 39 automated tests and verified end-to-end.

**Keywords:**

*penetration testing, agentic AI, LLM, Strix, RAG, Qdrant, Metasploit, RBAC, MFA, FastAPI, Next.js.*

## Table des matières

Clic droit → « Mettre à jour les champs ».

## Liste des figures

Clic droit → « Mettre à jour les champs ».

## Liste des tableaux

Clic droit → « Mettre à jour les champs ».

## Liste des abréviations

| Abréviation | Signification                                                         |
|-------------|-----------------------------------------------------------------------|
| LLM         | Large Language Model (grand modèle de langage)                        |
| RAG         | Retrieval-Augmented Generation (génération augmentée de récupération) |
| MSF         | Metasploit Framework                                                  |
| RBAC        | Role-Based Access Control (contrôle d'accès par rôles)                |
| MFA / TOTP  | Multi-Factor Authentication / Time-based One-Time Password            |
| CSRF        | Cross-Site Request Forgery                                            |
| API         | Application Programming Interface                                     |
| WS          | WebSocket                                                             |
| CVE         | Common Vulnerabilities and Exposures                                  |
| OCR         | Optical Character Recognition (reconnaissance optique de caractères)  |
| ORM         | Object-Relational Mapping                                             |
| HIL         | Human-In-the-Loop (validation humaine)                                |
| CI/CD       | Continuous Integration / Continuous Deployment                        |
| PoC         | Proof of Concept (preuve de concept)                                  |

## Introduction générale

La transformation numérique des organisations s'accompagne d'une surface d'attaque en expansion permanente. Les tests d'intrusion (penetration testing) constituent l'un des moyens les plus efficaces pour évaluer, de façon offensive et contrôlée, la robustesse d'un système d'information. Toutefois, réalisés manuellement, ils demeurent coûteux en temps, difficilement reproductibles et fortement dépendants de l'expertise de l'opérateur.

L'émergence des grands modèles de langage (LLM) et des architectures dites « agentiques » ouvre la voie à une automatisation intelligente de ce processus : des agents autonomes planifient, exécutent et enchaînent des actions offensives en s'appuyant sur des outils réels. Le moteur open-source Strix illustre cette approche. Néanmoins, deux limites subsistent : d'une part, la phase d'exploitation reste fragile lorsqu'elle est entièrement confiée au raisonnement du modèle, sujet aux hallucinations ; d'autre part, l'usage en équipe manque d'une interface d'opérations sécurisée offrant gouvernance, traçabilité et pilotage.

Ce projet répond à ces deux limites par une approche hybride et intégrée. Il ancre le raisonnement des agents dans un service de connaissances RAG spécialisé pour l'exploitation, privilégie un exécuteur éprouvé (Metasploit) au moyen d'un outil déterministe unique, puis conçoit et réalise une console web professionnelle, Strix Console, dédiée à l'orchestration et au suivi sécurisés des campagnes. Le résultat forme une chaîne cohérente allant de l'interface d'opérations jusqu'à l'exécution offensive dans le bac à sable.

Le présent rapport est organisé en six parties. La première pose le cadre du projet et l'état de l'art. La deuxième analyse le moteur agentique Strix. La troisième détaille le service de connaissances Pentest RAG. La quatrième présente l'intégration Metasploit + RAG qui fiabilise l'exploitation. La cinquième expose la conception, la réalisation et l'interface de la console web. La sixième présente les tests et l'évaluation. Une conclusion générale synthétise les apports et ouvre des perspectives.

# Partie I

*Contexte, problématique et état de l'art*

# Chapitre 1 — Cadre général du projet

## 1.1 Introduction

Ce chapitre situe le projet dans son contexte, formule la problématique, présente la solution proposée et décrit la méthodologie de gestion adoptée pour mener le développement.

## 1.2 Contexte et problématique

Le pentest manuel souffre de plusieurs limites structurelles : durée d'exécution élevée, dépendance à l'expertise humaine, faible reproductibilité et difficulté de traçabilité. Les plateformes d'automatisation par IA agentique, comme Strix, apportent une réponse prometteuse mais présentent, en l'état, deux insuffisances :

- **Exploitation fragile** — en confiant l'intégralité de la décision d'exploitation au LLM, l'agent peut échouer à sélectionner ou paramétrer correctement un exploit, faute d'accès structuré à une base de connaissances et à un arsenal éprouvé.
- **Absence d'interface d'opérations** — l'outil est piloté en ligne de commande, sans gestion des utilisateurs, sans contrôle d'accès, sans suivi temps réel ni gouvernance, ce qui limite l'usage collaboratif et la valeur opérationnelle en entreprise.

La problématique retenue peut ainsi se formuler : comment fiabiliser la phase d'exploitation d'un agent de pentest fondé sur un LLM, et comment en faire un service d'opérations de sécurité sûr, gouverné et exploitable en équipe ?

## 1.3 Solution proposée et contributions

La solution combine quatre briques en un système intégré. Les contributions propres de ce projet portent sur l'intégration (Metasploit + RAG) et sur la console web ; le moteur Strix et le service RAG sont mobilisés et adaptés au service de cet objectif.

| Brique           | Nature                                  | Apport dans le projet                                       |
|------------------|-----------------------------------------|-------------------------------------------------------------|
| Strix            | Moteur agentique open-source (mobilisé) | Orchestration multi-agent et exécution des outils offensifs |
| Pentest RAG      | Service de connaissances (conçu/adapté) | Ancrage du raisonnement dans un corpus offensif vérifié     |
| Metasploit       | Framework d'exploitation (intégré)      | Exécuteur éprouvé, vérification non destructive (check)     |
| exploit_research | Outil déterministe (contribution)       | Recommandation « module d'abord », décision codée en Python |
| Strix Console    | Console web (contribution)              | Sécurité, RBAC, temps réel,                                 |

|  |  |                          |
|--|--|--------------------------|
|  |  | gouvernance et reporting |
|--|--|--------------------------|

Tableau 1 — Briques du système et contributions du projet.

## 1.4 Objectifs

- Fiabiliser la phase d'exploitation en l'ancrant dans des connaissances vérifiées et un arsenal éprouvé.
- Rendre la décision d'exploitation déterministe et testable plutôt que laissée au seul modèle.
- Offrir une plateforme d'opérations sécurisée, gouvernée et collaborative.
- Assurer le pilotage et le suivi temps réel des campagnes, du lancement au reporting.
- Garantir la qualité par des tests automatisés et une vérification de bout en bout.

## 1.5 Méthodologie de gestion de projet

Le développement a suivi une démarche itérative et incrémentale, organisée en phases successives livrant chacune un incrément fonctionnel testé et vérifié. Cette approche, proche de Scrum/Kanban, a permis de sécuriser les fondations avant d'empiler les fonctionnalités, tout en conservant une application exécutable à chaque étape.

- **Intégration Metasploit + RAG** — outil exploit\_research, playbooks et tests.
- **Phase 0 (Fondations)** — couche de données asynchrone, migrations, files et harnais de tests.
- **Phase 1 (Authentification)** — cookies httpOnly, CSRF, MFA TOTP, RBAC, sessions, audit.
- **Phase 2 (Temps réel)** — WebSocket, réconciliation des exécutions orphelines.
- **Phase 3 (Sortie moteur)** — arbre d'agents, console live, télémétrie (jetons, coût).
- **Phase 4 (Post-exploitation)** — 5<sup>e</sup> phase du pipeline et événements typés.
- **Phase 5 (Domaine)** — triage, comparaison de scans, alertes, planification récurrente.
- **Phase 6 (Qualité)** — palette de commandes, observabilité, durcissement.

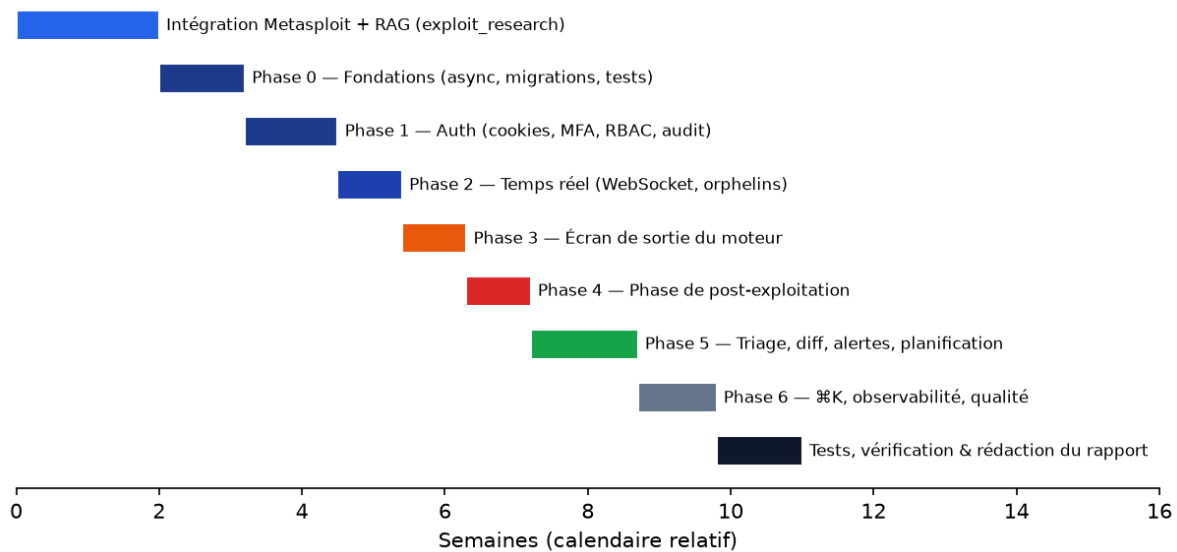


Figure 1 — Planning du projet (méthodologie itérative par phases).

## 1.6 Conclusion

Ce chapitre a défini le cadre, la problématique et la trajectoire du projet. Le chapitre suivant établit l'état de l'art sur lequel s'appuient les choix de conception.

## Chapitre 2 — État de l'art

### 2.1 Introduction

Ce chapitre présente les fondements des tests d'intrusion, les approches d'automatisation par IA agentique et les briques techniques mobilisées (Strix, RAG, Metasploit), afin de positionner la contribution du projet.

### 2.2 Tests d'intrusion et Red Teaming

Un test d'intrusion reproduit, de manière autorisée et méthodique, le comportement d'un attaquant afin d'identifier et de démontrer des vulnérabilités exploitables. Le cycle de vie standard comprend la reconnaissance, l'évaluation des vulnérabilités, l'exploitation, la post-exploitation puis la rédaction du rapport. Le Red Teaming étend cette démarche à une simulation d'adversaire plus réaliste et furtive.

### 2.3 Automatisation par IA agentique et LLM

Un système « agentique » orchestre un ou plusieurs agents fondés sur un LLM capables de raisonner, de décomposer une tâche, d'appeler des outils et d'itérer à partir des retours d'exécution. Appliqué au pentest, ce paradigme automatise la planification et l'enchaînement des actions offensives. Des travaux comme PentestGPT ou AutoGPT ont démontré la faisabilité de cette approche, tout en révélant ses limites : hallucinations, décisions d'exploitation peu fiables et besoin d'un ancrage dans des connaissances vérifiées.

### 2.4 Le paradigme RAG

La génération augmentée de récupération (RAG) répond directement au problème de l'hallucination : plutôt que de s'appuyer sur la seule mémoire interne du modèle, le système récupère, dans une base de connaissances de confiance, les fragments pertinents pour la requête, puis les fournit au modèle comme contexte d'ancrage. Dans un contexte offensif, ce mécanisme évite les CVE inexistantes et les payloads erronés, en fondant chaque réponse sur des documents réels (writeups, exploits, données CVE).

### 2.5 Briques mobilisées

- **Strix** — moteur multi-agent open-source de pentest ; un CLI côté hôte orchestre des agents qui exécutent des outils dans un bac à sable Docker (Kali).
- **Pentest RAG** — service auto-hébergé (FastAPI + Qdrant) qui indexe des connaissances d'exploitation et fournit, sur requête, un guidage structuré ancré dans des documents réels.
- **Metasploit Framework** — arsenal d'exploitation de référence ; ses modules apportent vérification non destructive (check), gestion des cas limites et sessions structurées.

La combinaison de ces briques permet d'ancrer le raisonnement de l'agent (RAG) tout en privilégiant un exécuteur éprouvé (Metasploit) lorsqu'un module existe, sous le pilotage d'une console d'opérations.

## **2.6 Positionnement du projet**

Par rapport à l'existant, la contribution est double : (i) rendre l'exploitation déterministe en collapsant le protocole RAG + Metasploit dans un outil unique dont la décision est codée en Python ; (ii) doter la plateforme d'une console d'opérations sécurisée, absente des outils comparables, apportant gouvernance, temps réel et traçabilité.

## **2.7 Conclusion**

L'état de l'art confirme la pertinence d'une approche hybride « IA agentique + connaissances vérifiées + arsenal éprouvé » et l'intérêt d'une interface d'opérations. Les parties suivantes détaillent chacune des briques, puis leur intégration.

# Partie II

*Le moteur agentique Strix*

## Chapitre 3 — Architecture et fonctionnement de Strix

### 3.1 Introduction

Strix est le socle offensif de la plateforme : c'est lui qui, in fine, exécute les actions d'attaque. Ce chapitre en présente l'architecture, le système multi-agent, la boîte à outils, la base de connaissances et le flux d'exécution.

### 3.2 Architecture technique

L'architecture distingue l'orchestration (côté hôte) de l'exécution des outils (dans un bac à sable isolé). Un CLI Python orchestre les agents et présente une interface en terminal ; un bac à sable Docker fondé sur une image Kali exécute les outils via un serveur d'outils ; un fournisseur LLM est interrogé à travers la bibliothèque LiteLLM.

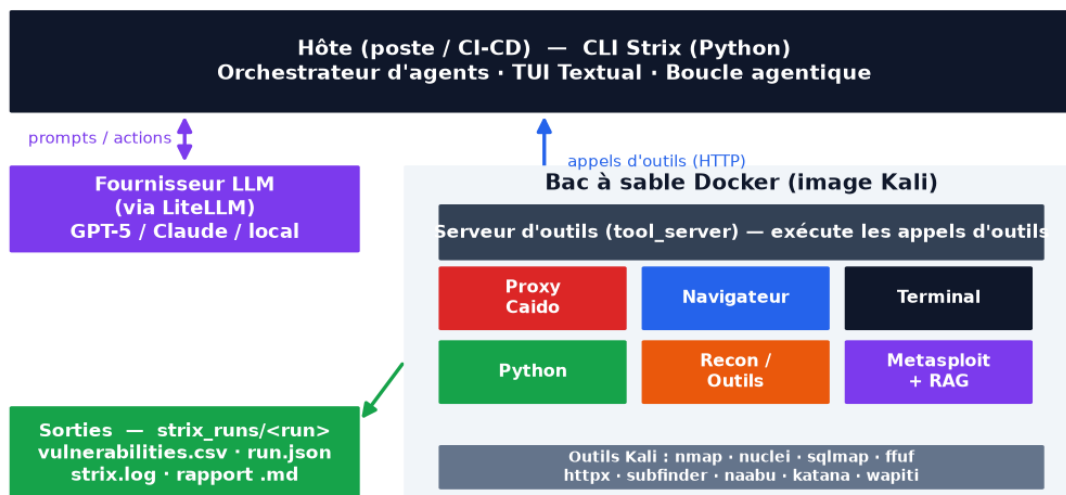


Figure 2 — Architecture technique du moteur Strix (hôte, bac à sable, LLM, sorties).

### 3.3 Système multi-agent (graphe d'agents)

Plutôt qu'un agent unique, un agent racine décompose la cible en tâches indépendantes et engendre dynamiquement des sous-agents spécialisés (reconnaissance, injection, contrôle d'accès, SSRF/XXE,

reporting...). Ces agents s'exécutent en parallèle, collaborent et partagent leurs découvertes ; des sous-agents de validation confirment l'exploitabilité par preuve de concept.

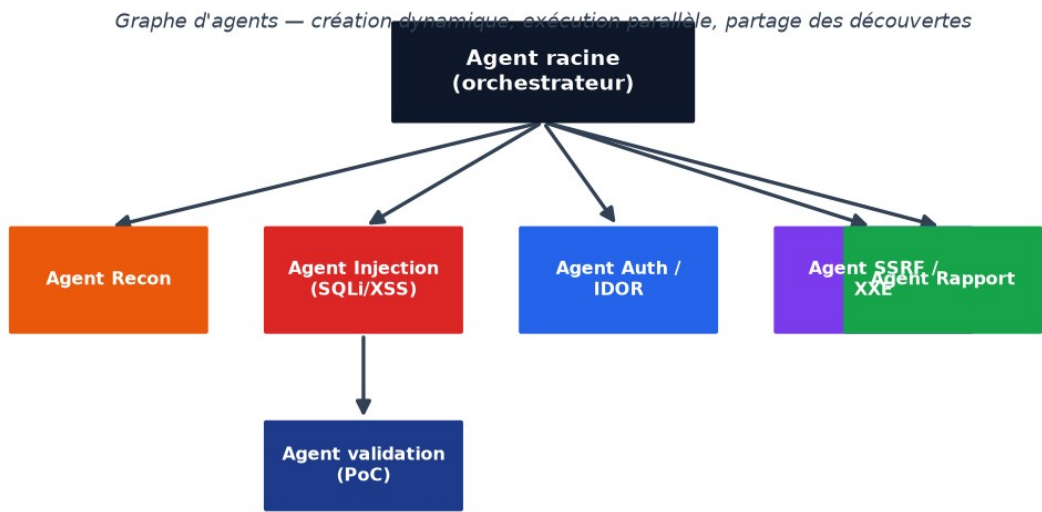


Figure 3 — Graphe d'agents : orchestration multi-agent de Strix.

### 3.4 Boîte à outils des agents

Les agents disposent d'un ensemble d'outils couvrant l'ensemble du cycle offensif, chacun exposé au LLM avec un schéma d'appel.

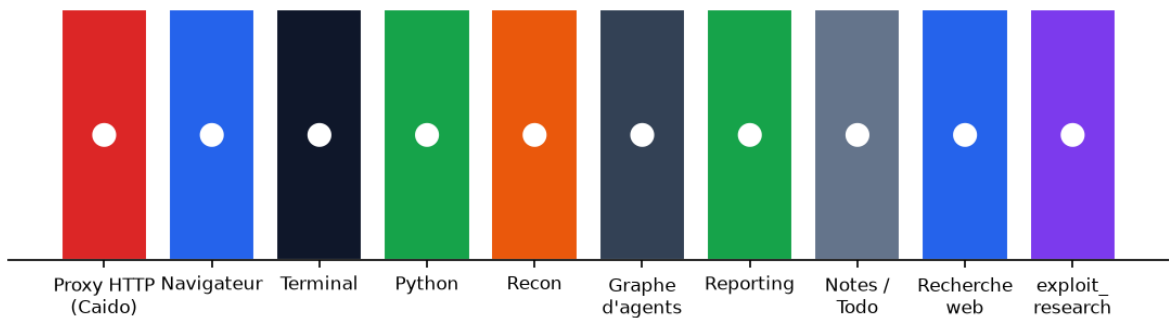


Figure 4 — Catégories d'outils des agents Strix.

| Outil         | Rôle                                              |
|---------------|---------------------------------------------------|
| proxy (Caído) | Proxy HTTP complet : interception et manipulation |

|                                 |                                                                         |
|---------------------------------|-------------------------------------------------------------------------|
|                                 | requêtes/réponses                                                       |
| <b>browser</b>                  | Automatisation d'un navigateur multi-onglets (XSS, CSRF, flux d'auth)   |
| <b>terminal</b>                 | Shells interactifs pour l'exécution de commandes                        |
| <b>python</b>                   | Runtime Python pour développer et valider des exploits                  |
| <b>agents_graph</b>             | Création et coordination des sous-agents                                |
| <b>reporting / notes / todo</b> | Documentation des vulnérabilités, gestion des découvertes et des tâches |
| <b>web_search</b>               | Recherche d'informations (CVE, techniques, PoC)                         |
| <b>exploit_research</b>         | (Contribution) RAG + Metasploit, recommandation « module d'abord »      |
| <b>finish</b>                   | Clôture d'un agent ou de la campagne                                    |

Tableau 2 — Principaux outils de la boîte à outils Strix.

### 3.5 Base de connaissances (skills)

Les « skills » sont des paquets de connaissances spécialisées injectés dynamiquement dans le prompt d'un agent selon sa tâche (jusqu'à cinq par agent). Ils couvrent notamment 17 classes de vulnérabilités, des frameworks, des technologies, des protocoles, le cloud, la reconnaissance et les modes de scan.

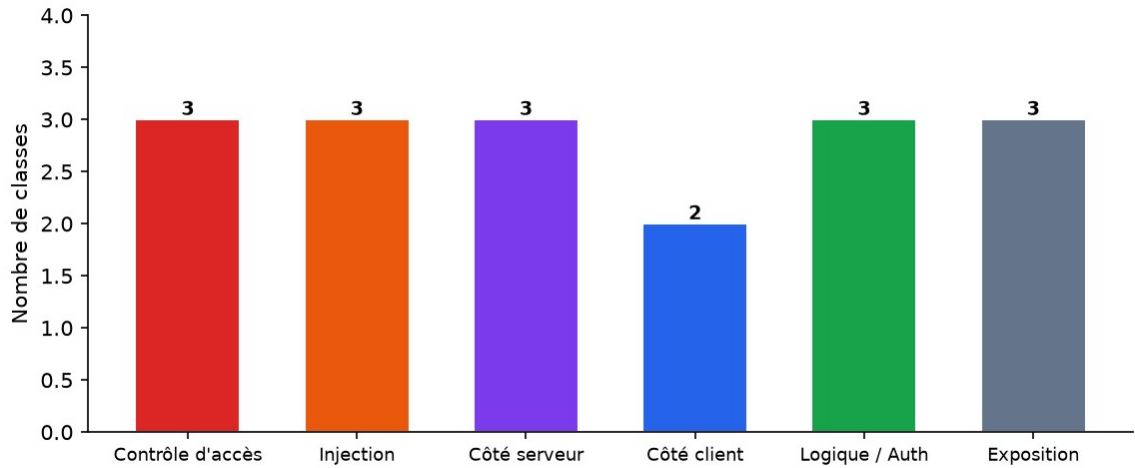


Figure 5 — Classes de vulnérabilités couvertes par Strix (regroupées par famille).

### 3.6 Flux d'exécution et livrables

Une campagne enchaîne reconnaissance, évaluation des vulnérabilités, exploitation, validation par PoC et reporting. Trois modes ajustent la profondeur (quick, standard, deep). Les résultats sont persistés dans un répertoire de campagne (strix\_runs/) : vulnerabilities.csv, fiches par vulnérabilité, run.json (télémétrie LLM), journal d'exécution et rapport de synthèse.



*Figure 6 — Flux d'exécution d'une campagne Strix.*

### 3.7 Conclusion

Strix fournit un moteur offensif puissant et extensible. Sa limite — la fragilité de l'exploitation confiée au modèle — motive l'ancrage par RAG et l'intégration de Metasploit, objets des parties suivantes.

# Partie III

*La base de connaissances Pentest RAG*

# Chapitre 4 — Conception du service Pentest RAG

## 4.1 Introduction

Le service Pentest RAG ancre le raisonnement des agents dans un corpus offensif vérifié. Ce chapitre présente son architecture conteneurisée, son pipeline en trois phases et son API.

## 4.2 Architecture technique

L'architecture est orchestrée par Docker Compose et réunit deux services sur un réseau privé : l'API applicative (FastAPI) et la base vectorielle Qdrant. Le backend LLM (Ollama en local ou OpenRouter en cloud) est sollicité en externe, et la persistance est assurée par des volumes.

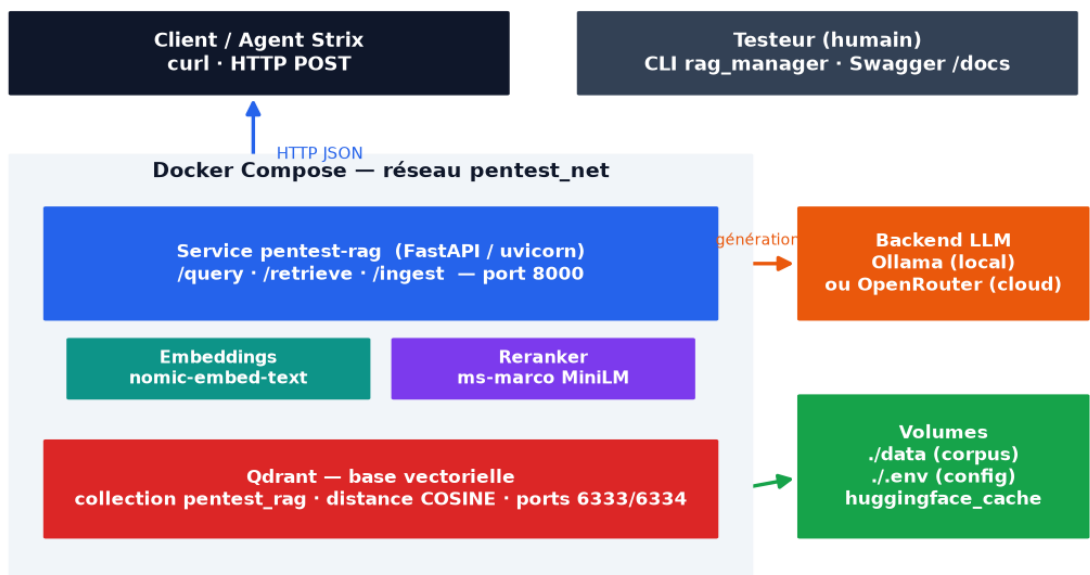


Figure 7 — Architecture technique du service Pentest RAG.

| Composant        | Technologie                               |
|------------------|-------------------------------------------|
| API applicative  | FastAPI + uvicorn (port 8000)             |
| Base vectorielle | Qdrant (ports 6333/6334, distance COSINE) |
| Embeddings       | nomic-ai/nomic-embed-text-v1              |
| Reranker         | cross-encoder/ms-marco-MiniLM-L-6-v2      |
| Backend LLM      | Ollama (local) / OpenRouter (cloud)       |

|                      |                                     |
|----------------------|-------------------------------------|
|                      |                                     |
| <b>Orchestration</b> | Docker Compose (réseau pentest_net) |

Tableau 3 — Composants du service RAG.

### 4.3 Pipeline en trois phases

Le cœur fonctionnel repose sur trois phases adossées à Qdrant : l'ingestion construit la base vectorielle ; la récupération retrouve les fragments pertinents ; la génération produit une réponse structurée en JSON.



Qdrant (base vectorielle) — socle partagé entre les trois phases

Figure 8 — Pipeline du RAG en trois phases (ingestion, récupération, génération).

#### 4.3.1 Ingestion

L'ingestion transforme des données brutes hétérogènes (CVE, writeups, exploits, images) en vecteurs indexés. Le pipeline nettoie le texte, extrait sélectivement les champs utiles du JSON CVE, applique l'OCR (Tesseract) aux images, filtre le bruit binaire et les fragments faibles, et déduplique à deux niveaux (fichier et chunk) par empreinte SHA-256. Les fragments (taille 500, chevauchement 200) sont vectorisés et insérés par lots robustes (réessais).

#### 4.3.2 Récupération et reclassement

La récupération combine expansion de requête, recherche sémantique large (top-100), reclassement par Cross-Encoder et scoring composite pondérant la pertinence sémantique, les correspondances lexicales, les signaux techniques (payload, CVE, curl...) et la correspondance au contexte cible (service, version, OS, findings). Les cinq meilleurs fragments forment le contexte.

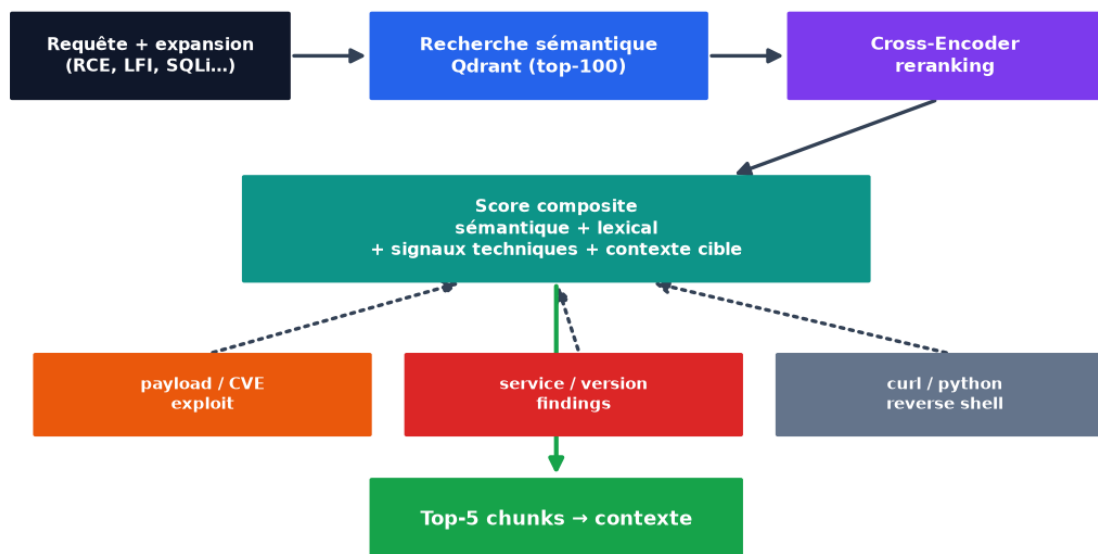


Figure 9 — Chaîne de récupération et de reclassement du RAG.

### 4.3.3 Génération structurée

Le modèle est contraint, par un prompt strict, à ne produire qu'un objet JSON conforme à un schéma imposé (StructuredRAGResponse), directement consommable par un agent.

| Champ                      | Description                                                         |
|----------------------------|---------------------------------------------------------------------|
| <b>vulnerability_id</b>    | Identifiant précis (CVE-XXXX-XXXX ou nom standard)                  |
| <b>reasoning</b>           | Explication technique de la faille et de son applicabilité          |
| <b>actionable_commands</b> | Commandes ordonnées de vérification et de déclenchement             |
| <b>payloads</b>            | Chaînes de payload brutes (curl, versions encodées, reverse shells) |
| <b>exploit_code</b>        | Script complet ou bloc de payload multi-lignes                      |
| <b>post_exploitation</b>   | Persistence et élévation de privilèges                              |
| <b>confidence_score</b>    | Score de fiabilité (0.0–1.0)                                        |

Tableau 4 — Schéma de la réponse structurée du RAG.

## 4.4 API REST

L'API expose trois endpoints : /retrieve (récupération rapide du contexte, sans LLM), /query (pipeline complet) et /ingest (indexation du corpus). Un cache en mémoire accélère les requêtes répétées.

## **4.5 Conclusion**

Le service RAG fournit un guidage d'exploitation fiable et structuré. Son intégration au moteur Strix, conjuguée à Metasploit, fait l'objet de la partie suivante.

# Partie IV

*Intégration Metasploit + RAG : fiabiliser l'exploitation*

## Chapitre 5 — L'outil déterministe exploit\_research

### 5.1 Introduction

Ce chapitre présente la contribution centrale du volet moteur : la fusion du protocole RAG + Metasploit en un unique outil déterministe, exploit\_research, et le moteur d'exploitation à trois niveaux qui en découle.

### 5.2 Principe : décision « module d'abord »

L'intégration transforme un protocole multi-étapes en un seul appel d'outil déterministe. Dès que la reconnaissance fournit un service, une version ou une CVE, l'agent invoque exploit\_research, qui : (1) interroge le RAG (/retrieve puis, après un portail de pertinence, /query) en contournant le proxy et avec un délai adapté (120 s) ; (2) recherche un module via msfconsole ; (3) détecte l'adresse LHOST du bac à sable ; (4) produit une recommandation déterministe privilégiant le module Metasploit lorsqu'il existe, et retombant sur le payload RAG sinon. La logique de décision est ainsi codée et testée, et non laissée au modèle.

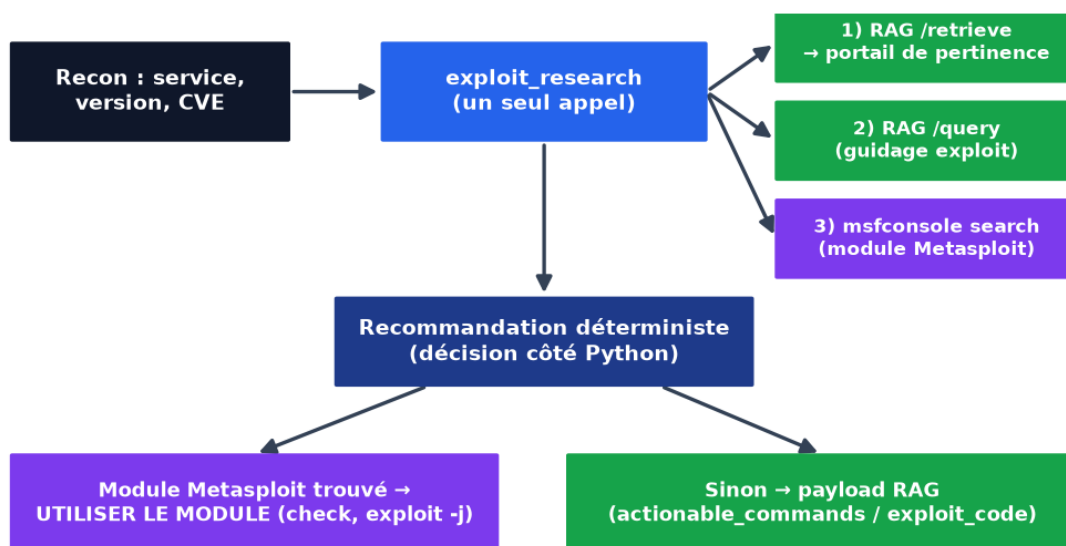


Figure 10 — Handoff d'exploitation : RAG puis Metasploit, avec priorité au module.

### 5.3 Moteur d'exploitation à trois niveaux

La recommandation est exécutée par un moteur à repli hiérarchique, qui choisit la voie la plus fiable disponible selon trois niveaux de priorité décroissante.

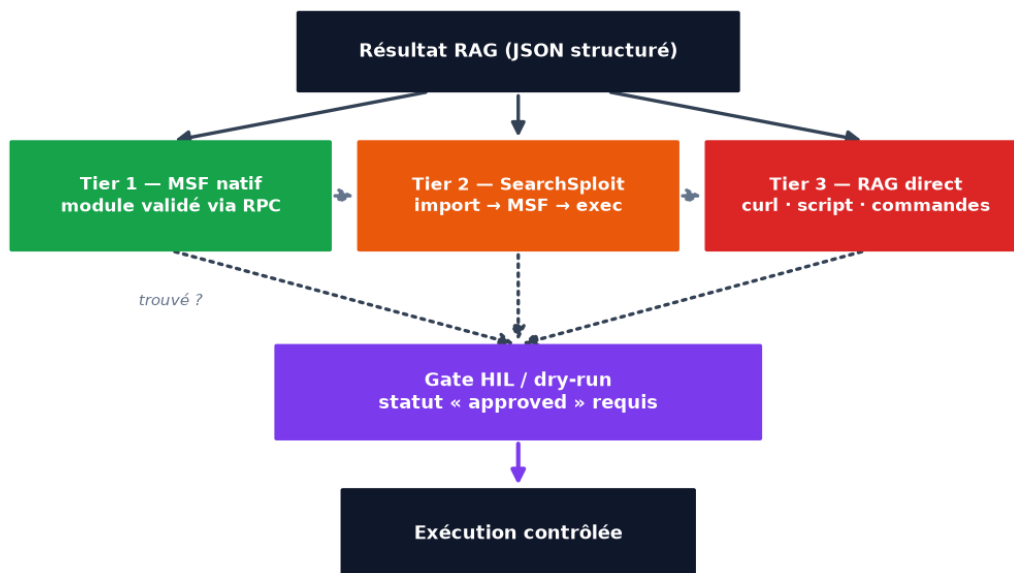


Figure 11 — Moteur d'exploitation à trois niveaux avec barrière de validation.

| Niveau                | Condition                                  | Action                                                            |
|-----------------------|--------------------------------------------|-------------------------------------------------------------------|
| Tier 1 — MSF natif    | Module présent dans Metasploit             | Validation puis exécution via MSF RPC                             |
| Tier 2 — SearchSploit | Module absent mais trouvé via SearchSploit | Import dans MSF, rechargement puis exécution                      |
| Tier 3 — RAG direct   | Ni MSF ni SearchSploit                     | Exécution des payloads HTTP (curl), du script, puis des commandes |

Tableau 5 — Les trois niveaux d'exécution de l'exploitation.

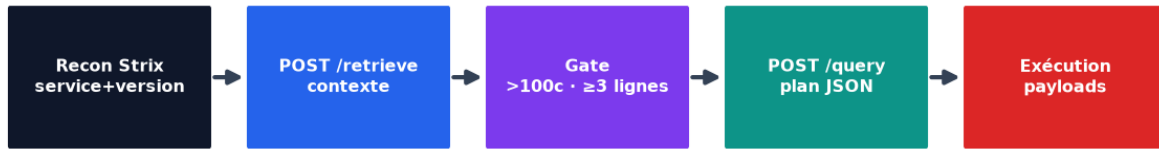
## 5.4 Barrière de validation humaine (HIL)

Quel que soit le niveau, aucun exploit n'est exécuté sans franchir une barrière de validation. Chaque plan démarre au statut « en attente d'approbation » et doit passer explicitement au statut « approuvé » avant toute exécution ; à défaut, il est bloqué. Ce garde-fou human-in-the-loop garantit qu'aucune action offensive n'est déclenchée de façon incontrôlée, et constitue un élément clé de la sécurité et de l'éthique du dispositif.

## 5.5 Protocole d'intégration côté agent

Côté agent, le contrat d'appel est explicite : après reconnaissance, l'agent appelle d'abord /retrieve, applique un portail de pertinence structurel (contexte de plus de 100 caractères et d'au moins 3 lignes

non vides), puis n'appelle /query que si le portail est franchi. Le champ `confidence_score` guide la décision : exploiter ( $> 0,7$ ), vérifier d'abord ( $0,4-0,7$ ) ou approfondir la reconnaissance ( $< 0,4$ ).



*confidence\_score :  $>0.7$  exploiter ·  $0.4-0.7$  vérifier ·  $<0.4$  recon · timeout 120 s*

*Figure 12 — Protocole d'intégration RAG côté agent Strix.*

```
# Construction de la commande du moteur (extrait)
cmd = ["strix", "--target", target, "--non-interactive", "--scan-mode", mode]
if instruction:
    cmd += ["--instruction", instruction]
```

## 5.6 Conclusion

L'intégration déplace la décision d'exploitation du modèle vers du code vérifiable et privilégie un exécuter éprouvé, tout en conservant un repli fondé sur le RAG. La partie suivante présente la console qui pilote et gouverne l'ensemble.

# Partie V

*La console web d'opérations — Strix Console*

# Chapitre 6 — Conception et architecture de la console

## 6.1 Introduction

Ce chapitre décrit l'architecture globale de la console, les besoins fonctionnels et non fonctionnels, les diagrammes UML, le modèle de données et la conception de la sécurité.

## 6.2 Architecture globale

La plateforme adopte une architecture trois tiers. Le client (Next.js) communique en même origine avec un serveur Next.js qui relaie les appels /api/\* vers un backend FastAPI asynchrone. Ce dernier expose les services métier, persiste les données dans PostgreSQL (SQLite en développement) et pilote le moteur Strix, lequel s'appuie sur Metasploit et le service RAG. Redis fournit files et cache.

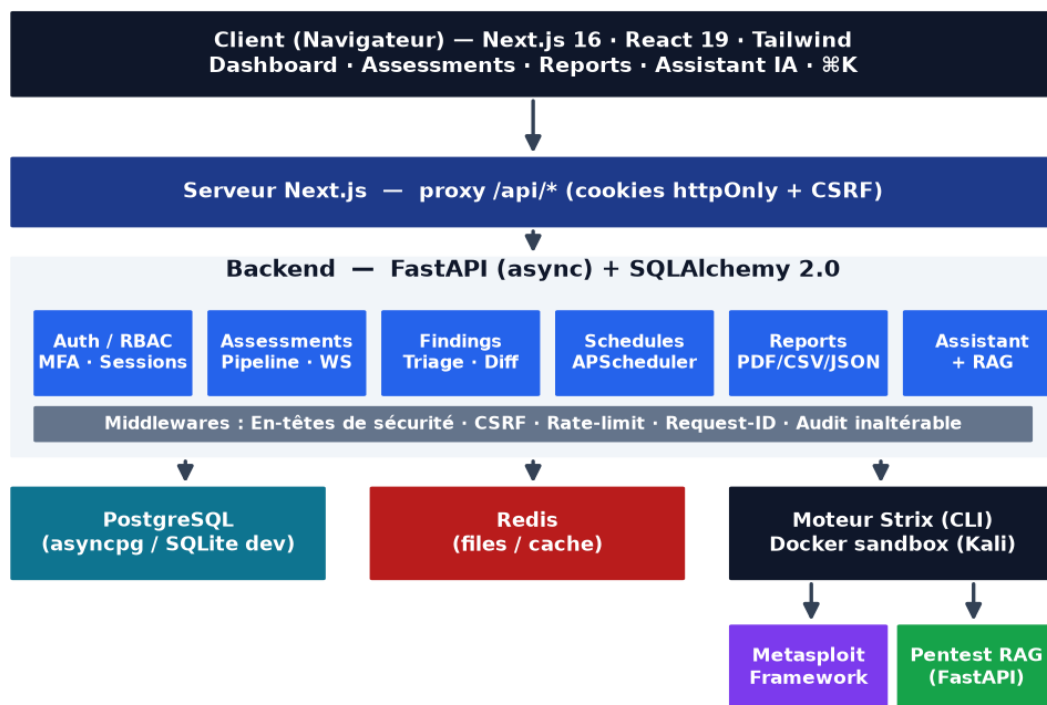


Figure 13 — Architecture globale de la plateforme Strix Console.

## 6.3 Besoins

### 6.3.1 Acteurs

Quatre rôles système sont définis, du moins au plus privilégié : Observateur (lecture seule), Analyste (création et exécution de tests, triage, export), Manager (supervision de toutes les campagnes, audit, lecture des utilisateurs) et Administrateur (accès complet).

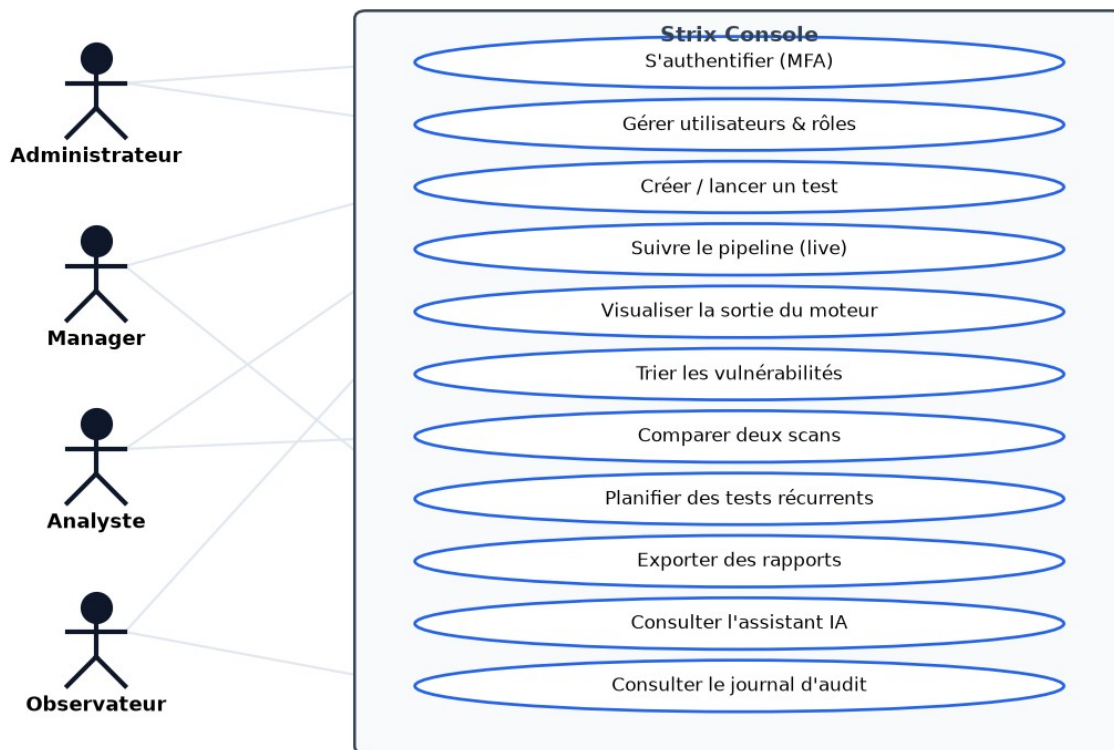


Figure 14 — Diagramme de cas d'utilisation de la plateforme.

### 6.3.2 Besoins fonctionnels

| Réf. | Besoin fonctionnel                                                   |
|------|----------------------------------------------------------------------|
| BF1  | S'authentifier avec cookies sécurisés et second facteur (MFA TOTP)   |
| BF2  | Gérer les utilisateurs, les rôles et les permissions (RBAC)          |
| BF3  | Créer, lancer, annuler et supprimer un test d'intrusion              |
| BF4  | Suivre en temps réel le pipeline (état, phases, journal)             |
| BF5  | Visualiser la sortie du moteur (arbre d'agents, télémétrie, console) |
| BF6  | Trier les vulnérabilités (statut, gravité, assignation)              |
| BF7  | Comparer deux scans d'une même cible (nouveaux / corrigés)           |
| BF8  | Planifier des tests récurrents (cron)                                |
| BF9  | Exporter des rapports (PDF, CSV, JSON)                               |

|             |                                                |
|-------------|------------------------------------------------|
|             |                                                |
| <b>BF10</b> | Consulter l'assistant IA et le journal d'audit |

Tableau 6 — Besoins fonctionnels principaux.

### 6.3.3 Besoins non fonctionnels

| Réf.        | Besoin non fonctionnel                                                 |
|-------------|------------------------------------------------------------------------|
| <b>BNF1</b> | Sécurité : jetons httpOnly, protection CSRF, MFA, en-têtes de sécurité |
| <b>BNF2</b> | Performance : backend asynchrone, temps réel via WebSocket             |
| <b>BNF3</b> | Traçabilité : journal d'audit inaltérable (chaîné par hachage)         |
| <b>BNF4</b> | Fiabilité : réconciliation des exécutions, tests automatisés           |
| <b>BNF5</b> | Maintenabilité : migrations versionnées, séparation claire des couches |
| <b>BNF6</b> | Observabilité : identifiants de requête, sondes /readyz et /livez      |

Tableau 7 — Besoins non fonctionnels.

## 6.4 Diagramme de séquence — exécution d'un test

Le scénario nominal d'exécution d'un test illustre l'enchaînement entre l'utilisateur, le frontend, l'API, le moteur et la base de données, ainsi que la diffusion des événements temps réel via WebSocket.

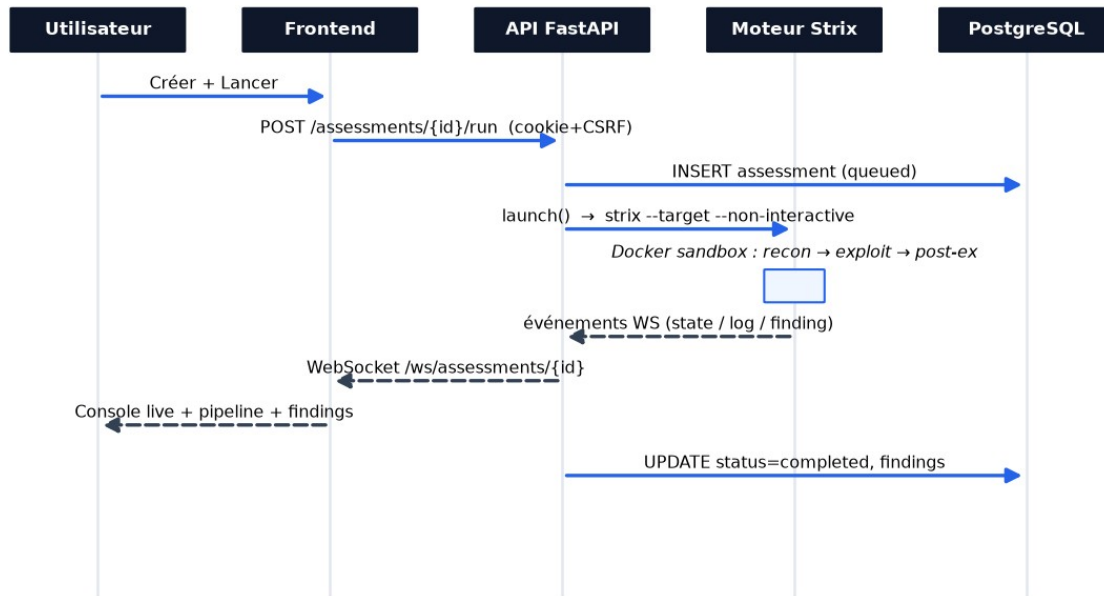


Figure 15 — Diagramme de séquence : exécution d'un test.

## 6.5 Modèle de données

Le modèle relationnel s'organise autour des entités suivantes, gérées via l'ORM SQLAlchemy 2.0 et versionnées par Alembic.

| Table                                   | Rôle                                                         |
|-----------------------------------------|--------------------------------------------------------------|
| users / roles / role_permissions        | Comptes, rôles et permissions (RBAC)                         |
| user_sessions                           | Sessions serveur (rotation, détection de réutilisation)      |
| assessments / findings                  | Campagnes et vulnérabilités découvertes                      |
| post_ex_events                          | Événements de post-exploitation (session, privesc, exfil...) |
| schedules                               | Définitions de tests récurrents (cron)                       |
| log_entries / audit_events              | Journaux d'exécution et journal d'audit chaîné               |
| reports / chat_sessions / chat_messages | Rapports et conversations de l'assistant                     |

Tableau 8 — Modèle de données (principales tables).

## 6.6 Conception de la sécurité

La sécurité repose sur une authentification par cookies httpOnly (jeton d'accès non lisible en JavaScript), une protection CSRF par double soumission, un second facteur TOTP obligatoire pour les rôles à

privilèges, une rotation des jetons de rafraîchissement avec détection de réutilisation, ainsi qu'un contrôle d'accès par rôles appliqué à chaque endpoint.

|             | stats:read | assessment:read | assessment:create/run | finding:triage | schedule:manage | report:export | assessment:read:all | user:read | audit:read | user:manage | role:manage | settings:manage |
|-------------|------------|-----------------|-----------------------|----------------|-----------------|---------------|---------------------|-----------|------------|-------------|-------------|-----------------|
| Observateur | ✓          | ✓               |                       |                |                 |               |                     |           |            |             |             |                 |
| Analyste    | ✓          | ✓               | ✓                     | ✓              | ✓               | ✓             |                     |           |            |             |             |                 |
| Manager     | ✓          | ✓               | ✓                     | ✓              | ✓               | ✓             | ✓                   | ✓         | ✓          |             |             |                 |
| Admin       | ✓          | ✓               | ✓                     | ✓              | ✓               | ✓             | ✓                   | ✓         | ✓          | ✓           | ✓           | ✓               |

Figure 16 — Matrice RBAC : permissions accordées par rôle système.

## 6.7 Conclusion

La conception établit une base sûre et modulaire. Le chapitre suivant décrit les technologies retenues et la réalisation.

## Chapitre 7 — Technologies et réalisation

### 7.1 Pile technologique

| Couche        | Technologies                                                          |
|---------------|-----------------------------------------------------------------------|
| Frontend      | Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS, Recharts |
| Backend       | FastAPI (async), SQLAlchemy 2.0, Pydantic v2, Uvicorn                 |
| Sécurité      | Argon2, PyJWT, pyotp (TOTP), slowapi (rate-limit)                     |
| Données       | PostgreSQL (asyncpg) / SQLite (aiosqlite), Alembic, Redis             |
| Planification | APScheduler, croniter                                                 |
| Moteur        | Strix (CLI multi-agent), Docker, Kali Linux, Metasploit, RAG          |
| Qualité       | pytest, ruff, GitHub Actions (CI)                                     |

Tableau 9 — Pile technologique du projet.

### 7.2 Réalisation du backend

- **Authentification & RBAC** — connexion par cookies, MFA TOTP, sessions serveur avec rotation et détection de réutilisation, permissions resource:action vérifiées par dépendance.
- **Pipeline d'évaluation** — création, lancement, annulation ; exécution en tâche de fond et diffusion d'événements d'état, de journaux et de vulnérabilités.
- **Temps réel** — bus d'événements interne pontant le runner vers les abonnés WebSocket ; réconciliation au démarrage des exécutions orphelines.
- **Triage & comparaison** — mise à jour du statut, de la gravité et de l'assignation ; comparaison des vulnérabilités entre deux scans d'une même cible.
- **Planification** — APScheduler déclenche des tests récurrents selon une expression cron.
- **Observabilité** — middleware d'identifiant de requête, sondes /readyz et /livez ; journal d'audit inaltérable chaîné par hachage.

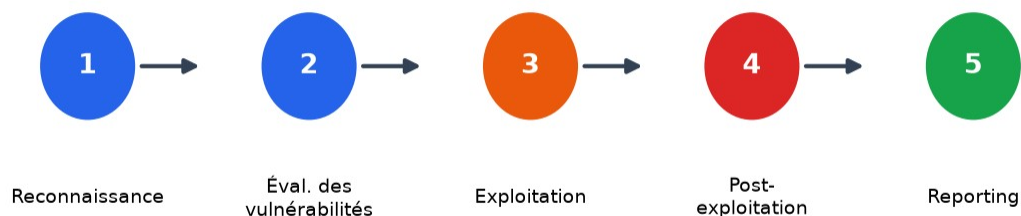


Figure 17 — Pipeline d'évaluation en 5 phases (dont la post-exploitation).

### 7.3 Réalisation du frontend

Le frontend, en Next.js (App Router), propose des modules dédiés : tableau de bord, campagnes et vue de détail (pipeline live, console, post-exploitation, triage, comparaison), rapports, journaux, audit, utilisateurs, rôles, planification, assistant IA et paramètres. Une palette de commandes offre une navigation rapide filtrée par permissions. L'interface est responsive, thématizable (clair/sombre) et cohérente grâce à un système de composants réutilisables.

### 7.4 Conclusion

La réalisation couvre l'ensemble des besoins, du moteur enrichi à la console d'opérations. Le chapitre suivant présente l'interface réalisée.

## Chapitre 8 — Présentation de l'interface réalisée

### 8.1 Introduction

Ce chapitre illustre, par des captures d'écran réelles, les principales vues de la console Strix Console telle qu'elle a été réalisée et vérifiée.

### 8.2 Authentification et tableau de bord

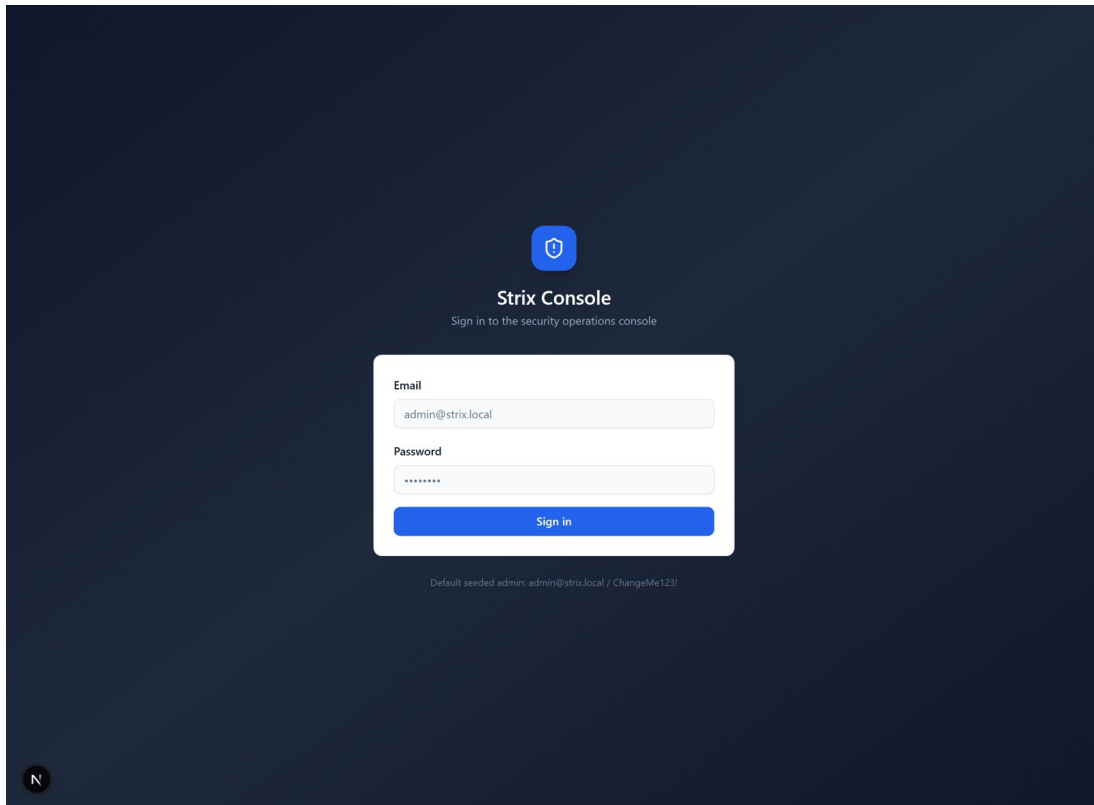


Figure 18 — Page de connexion sécurisée (cookies httpOnly, étape MFA).

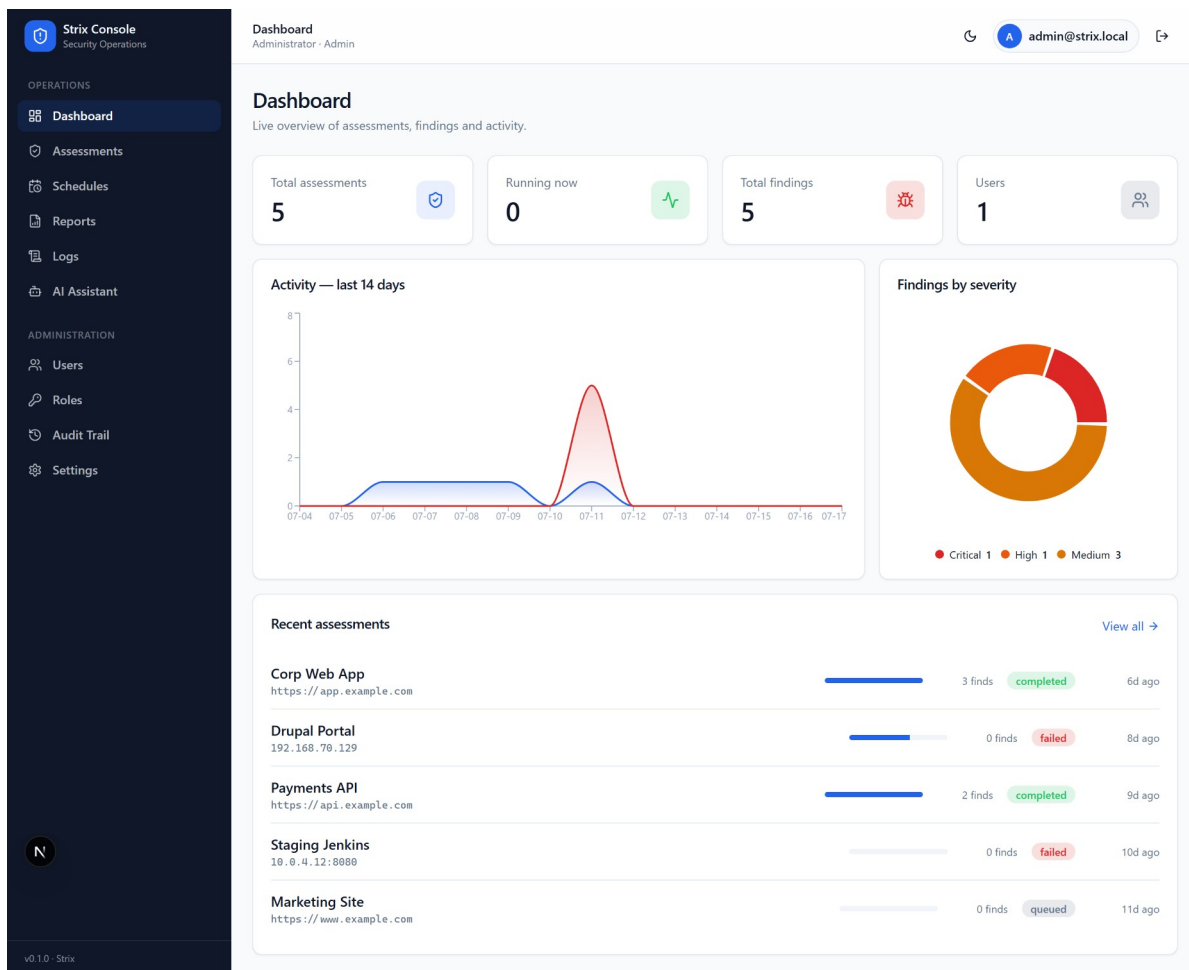


Figure 19 — Tableau de bord : statistiques, activité et répartition des vulnérabilités.

## 8.3 Gestion et détail des campagnes

Strix Console  
Security Operations

OPERATIONS

Dashboard

Assessments

Schedules

Reports

Logs

AI Assistant

ADMINISTRATION

Users

Roles

Audit Trail

Settings

N  
Virus - Strix

Assessments  
Administrator - Admin

admin@strix.local

+ New assessment

Search name or target...

| Name            | Target                  | Status    | Phase           | Progress | Findings | Created                |
|-----------------|-------------------------|-----------|-----------------|----------|----------|------------------------|
| Corp Web App    | https://app.example.com | completed | Reporting       | 100%     | 3        | Jul 11, 2026, 01:06 AM |
| Drupal Portal   | 192.168.70.129          | failed    | Exploitation    | 62%      | 0        | Jul 9, 2026, 10:06 PM  |
| Payments API    | https://api.example.com | completed | Reporting       | 100%     | 2        | Jul 8, 2026, 07:06 PM  |
| Staging Jenkins | 10.0.4.12:8080          | failed    | Vuln-Assessment | 0%       | 0        | Jul 7, 2026, 04:06 PM  |
| Marketing Site  | https://www.example.com | queued    | Queued          | 0%       | 0        | Jul 6, 2026, 01:06 PM  |

Figure 20 — Liste des campagnes de tests d'intrusion.

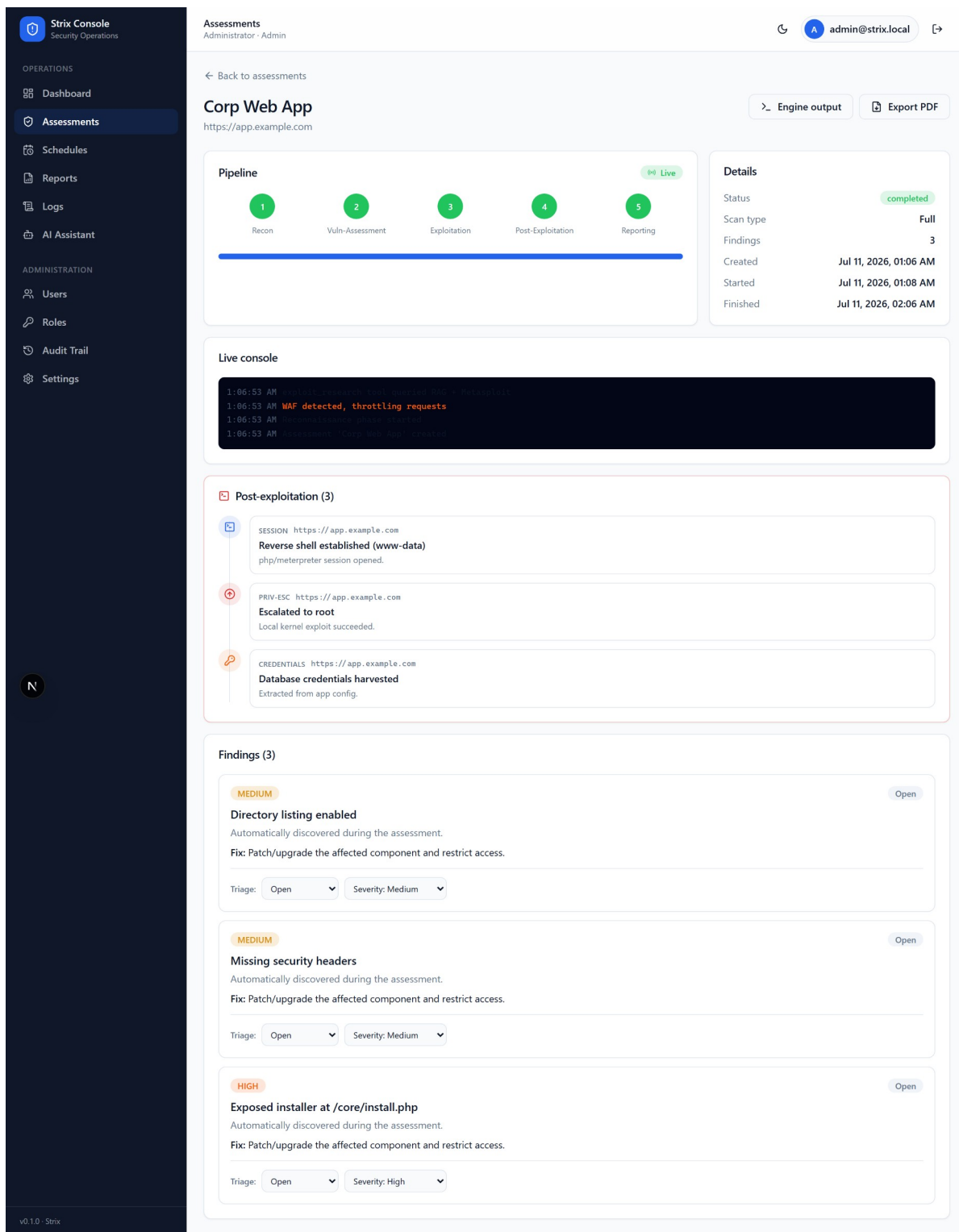


Figure 21 — Détail d'une campagne : pipeline en 5 phases, post-exploitation et triage des vulnérabilités.

## 8.4 Sortie du moteur en temps réel

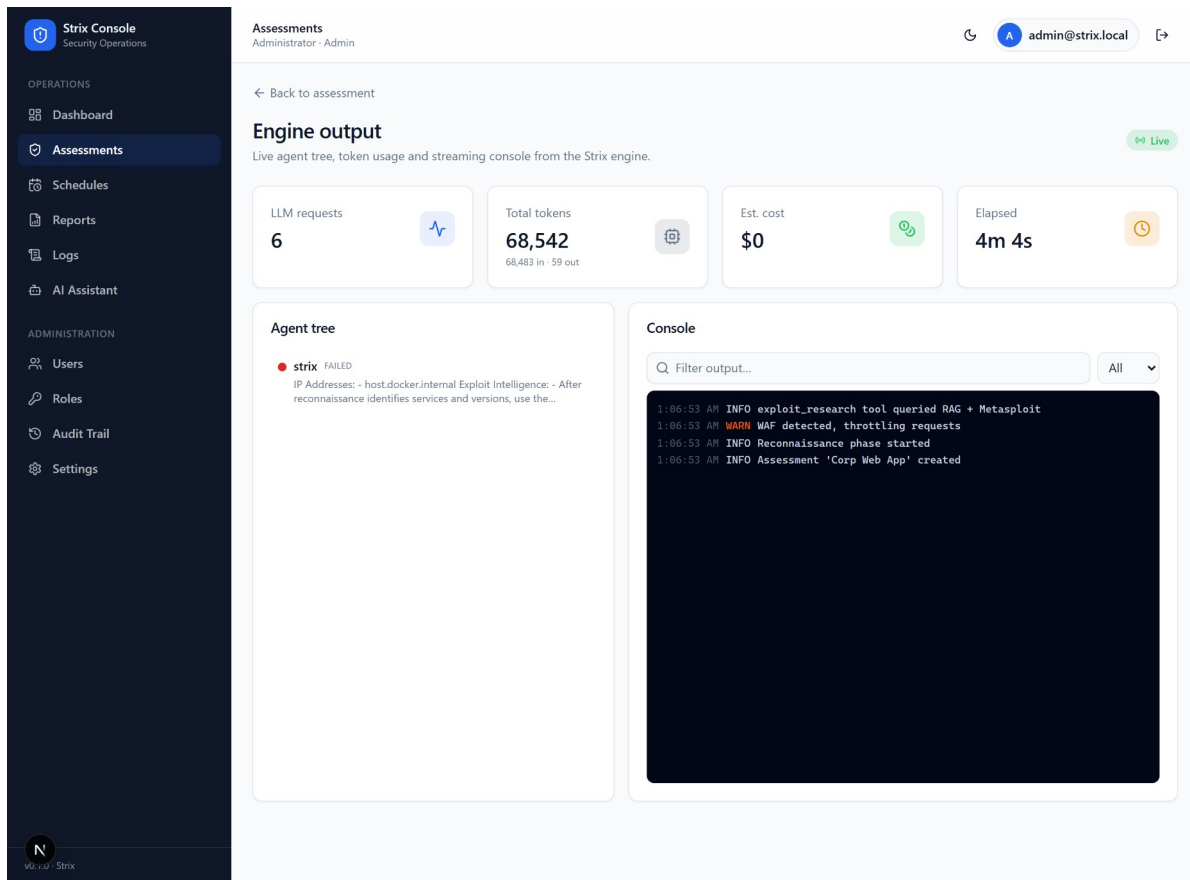
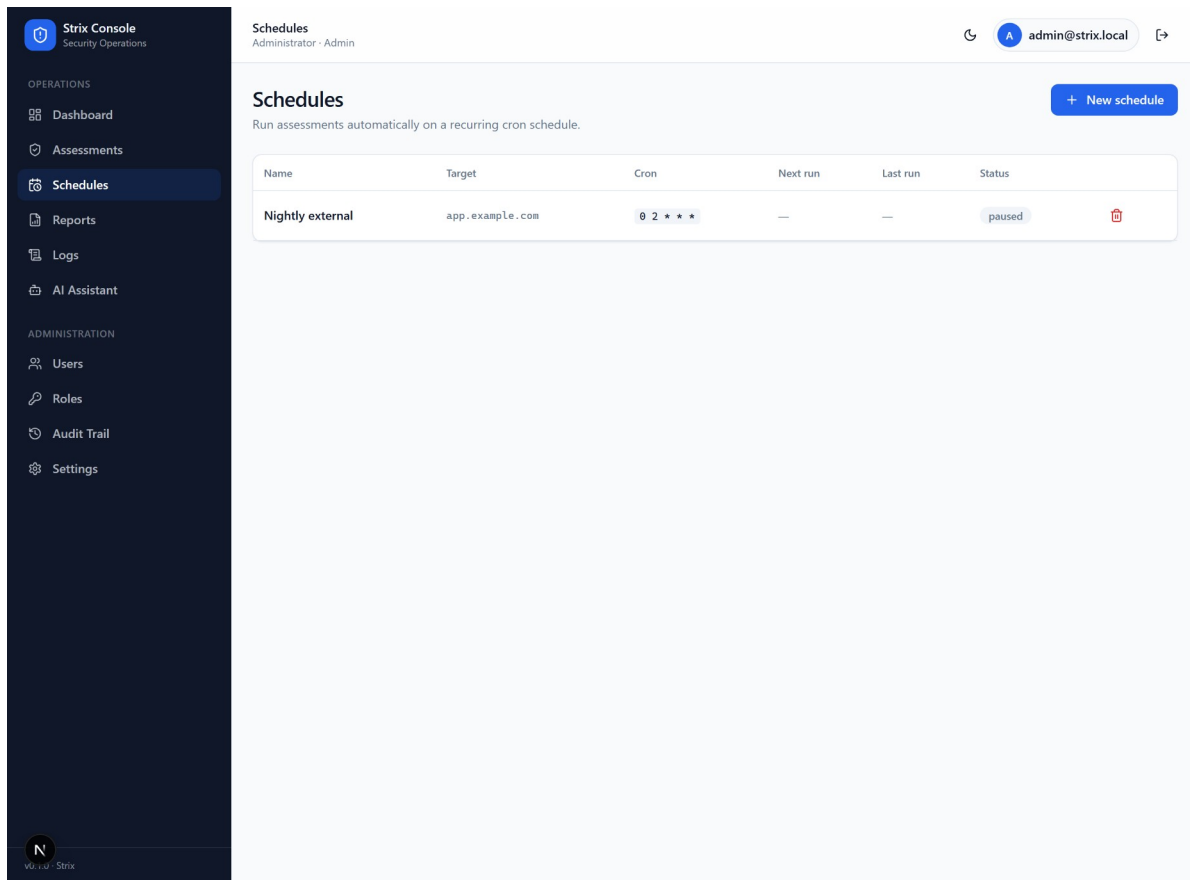


Figure 22 — Écran de sortie du moteur : arbre d'agents, télémétrie (jetons, coût) et console live.

## 8.5 Planification et gouvernance



The screenshot displays the 'Schedules' page in the Strix Console. The left sidebar contains navigation links for 'Dashboard', 'Assessments', 'Schedules' (selected), 'Reports', 'Logs', 'AI Assistant', 'Users', 'Roles', 'Audit Trail', and 'Settings'. The main content area shows a table of schedules. A single schedule, 'Nightly external', is listed with a target of 'app.example.com', a cron expression of '0 2 \* \* \*', and a status of 'paused'. A '+ New schedule' button is located in the top right corner of the main content area.

| Name             | Target          | Cron      | Next run | Last run | Status |
|------------------|-----------------|-----------|----------|----------|--------|
| Nightly external | app.example.com | 0 2 * * * | —        | —        | paused |

Figure 23 — Planification de tests récurrents (expression cron, prochaine exécution).

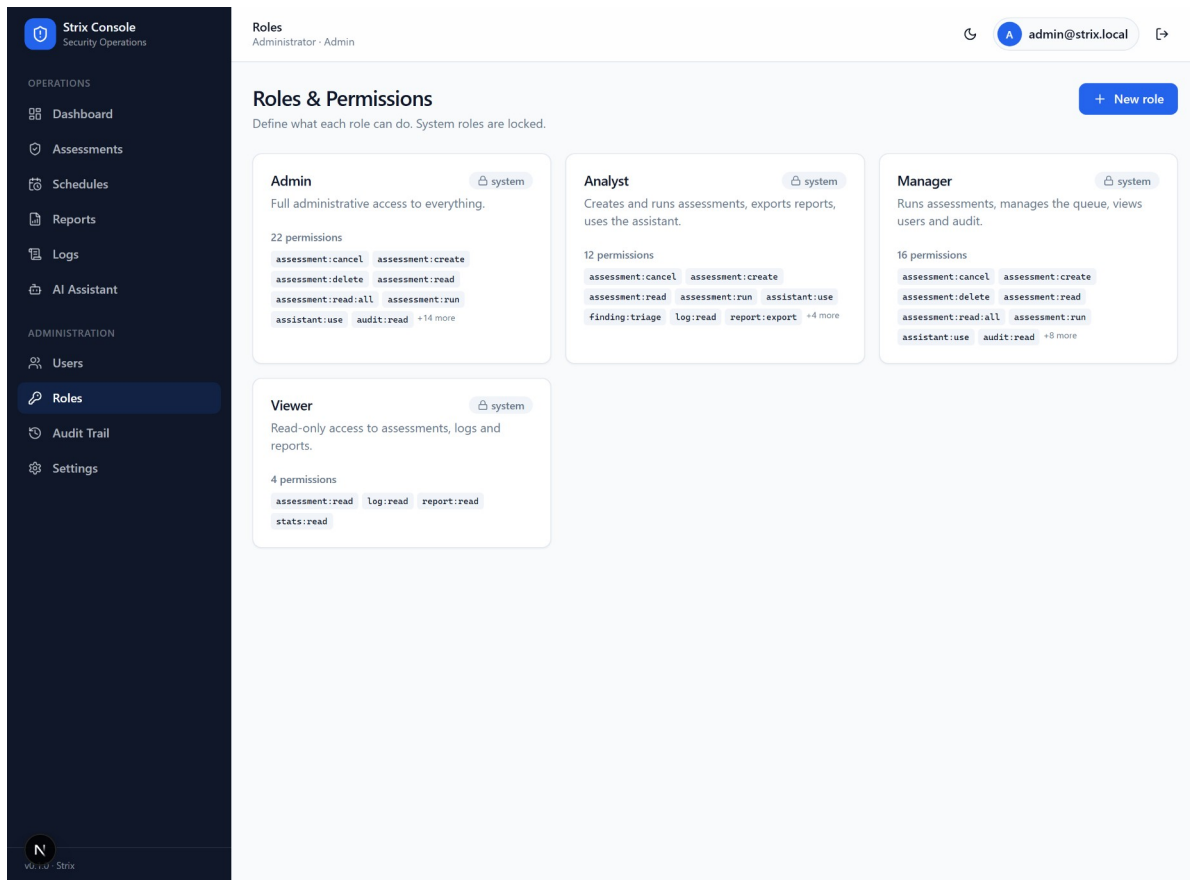


Figure 24 — Gestion des rôles et des permissions (RBAC).

## 8.6 Conclusion

L'interface concrétise l'ensemble des besoins fonctionnels dans une expérience cohérente, sécurisée et professionnelle. Le chapitre suivant en évalue la qualité.

# Partie VI

*Tests, évaluation et perspectives*

## Chapitre 9 — Tests et évaluation

### 9.1 Environnement de test

Les essais ont été menés sur un poste Windows 11 avec Docker Desktop, l'intégration ayant été validée sur une image de bac à sable Kali contenant Metasploit, contre une cible vulnérable de type application Drupal. Le backend a été exécuté en mode asynchrone (SQLite en développement, PostgreSQL en cible), le frontend via le serveur de développement Next.js.

### 9.2 Validation fonctionnelle

Chaque incrément a fait l'objet d'une vérification de bout en bout : connexion par cookies et MFA, application effective du RBAC (retour 403 pour un observateur sur une action d'administration), exécution et suivi temps réel d'une campagne, visualisation de la sortie du moteur avec télémétrie réelle (par exemple 6 requêtes LLM et 68 542 jetons sur une exécution), triage d'une vulnérabilité, comparaison de scans, planification récurrente et export de rapports. Côté intégration moteur, exploit\_research a produit, sur une cible Drupal, une recommandation ancrée (CVE-2018-7600, Drupalgeddon2) avec sélection du module Metasploit.

### 9.3 Couverture de tests

La plateforme est couverte par un harnais de tests automatisés (pytest) exécuté en intégration continue, complété par des vérifications navigateur. L'ensemble des tests est au vert.

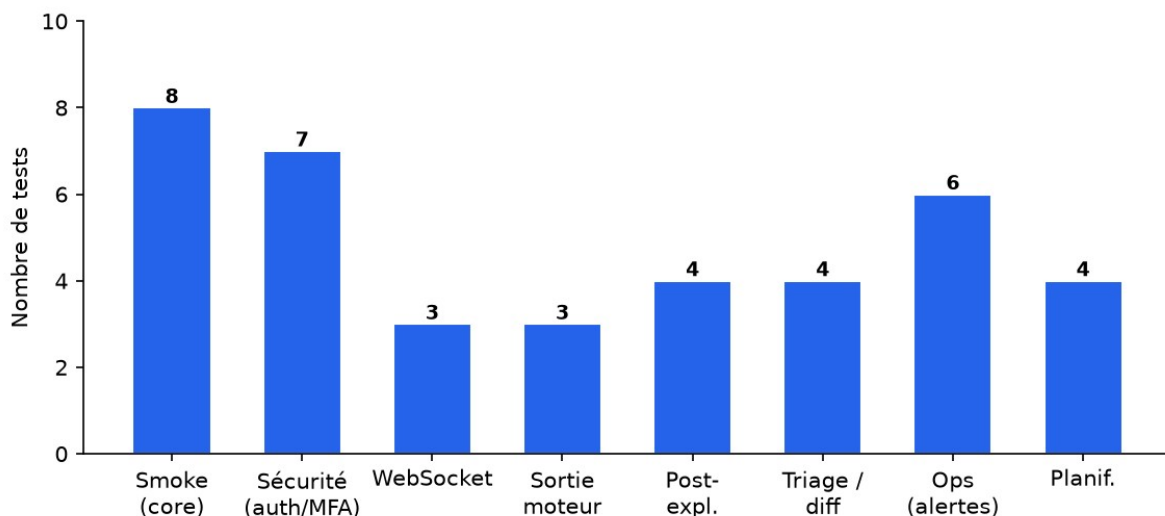


Figure 25 — Couverture de tests par module (39 tests, 100 % réussis).

## 9.4 Résultats et analyse comparative

À titre illustratif, la répartition des vulnérabilités par gravité sur une campagne de démonstration est présentée ci-dessous, suivie d'une analyse comparative qualitative entre un pentest manuel et l'usage de la plateforme.

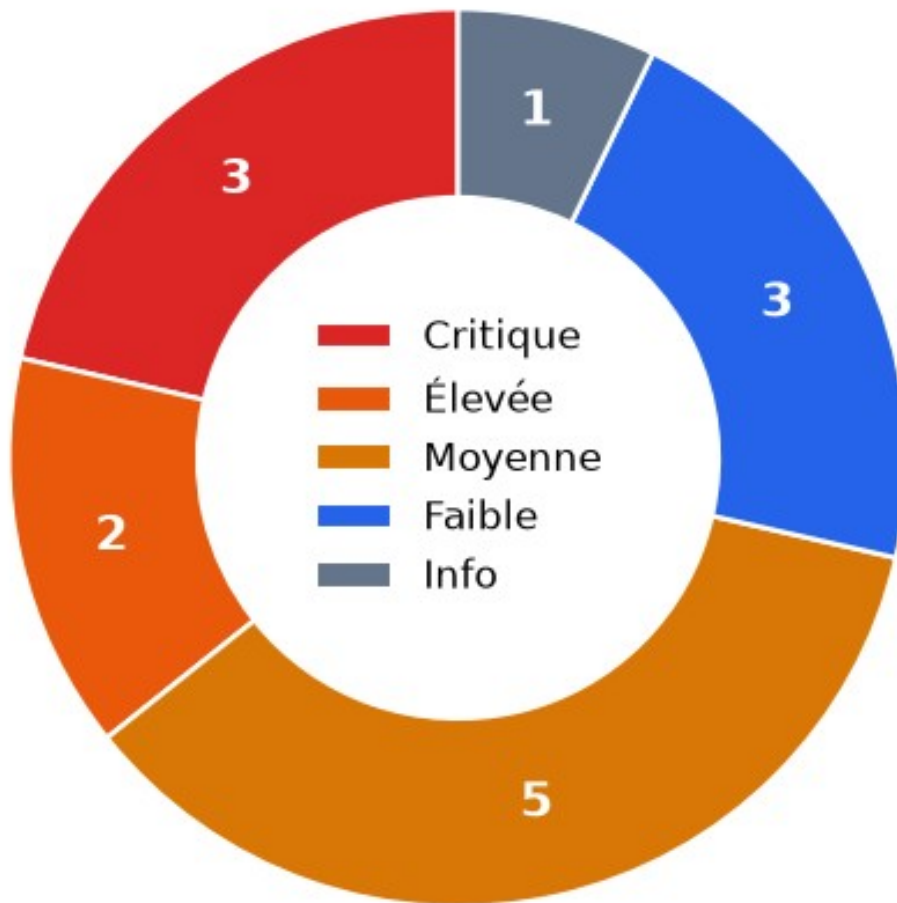


Figure 26 — Répartition des vulnérabilités par sévérité (campagne de démonstration).

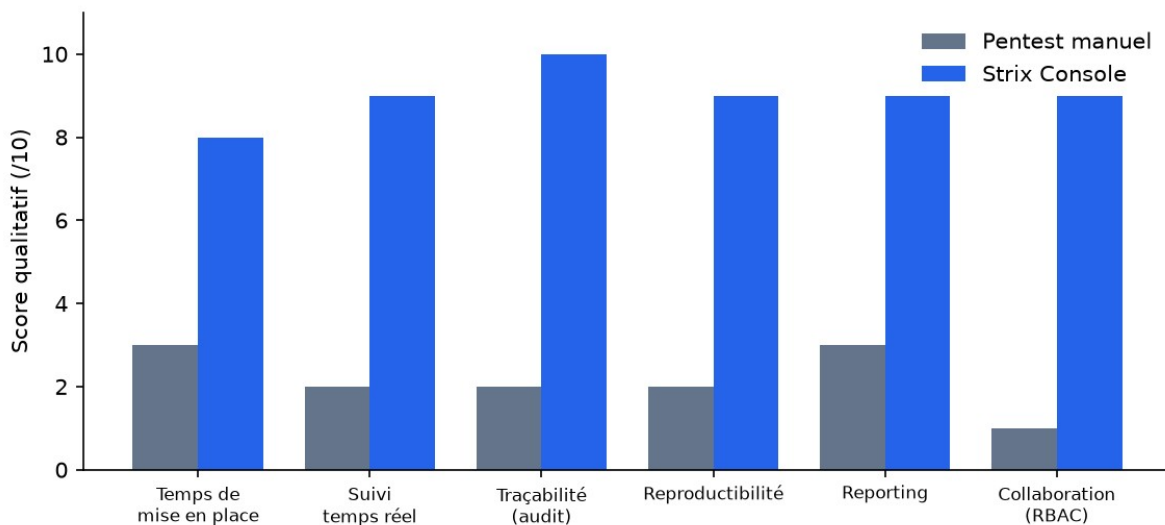


Figure 27 — Analyse comparative : approche manuelle vs plateforme.

| Critère                 | Résultat                                                 |
|-------------------------|----------------------------------------------------------|
| Tests automatisés       | 39 tests, 100 % réussis                                  |
| Analyse statique (ruff) | Aucune erreur                                            |
| Build frontend          | 14+ routes compilées, 0 vulnérabilité npm                |
| Sécurité                | Cookies httpOnly, CSRF, MFA, RBAC, audit chaîné vérifiés |
| Temps réel              | WebSocket : état, journaux et vulnérabilités diffusés    |
| Intégration moteur      | exploit_research validé (RAG + module Metasploit)        |

Tableau 10 — Synthèse des résultats de validation.

## 9.5 Limites et perspectives

- **Ordonnanceur mono-processus** — APScheduler s'exécute en processus unique ; un déploiement multi-workers nécessiterait un processus d'ordonnancement dédié.
- **Runner en thread** — l'exécution des scans s'appuie sur un thread interne ; une extraction vers un worker dédié améliorerait la durabilité.
- **Couverture du corpus RAG** — la qualité des réponses dépend de la fraîcheur et de l'étendue des connaissances ingérées.
- **Perspectives** — pièces jointes de preuve, scans multi-cibles, intégrations (Jira, SIEM), cartographie CVSS/CWE/OWASP, client TypeScript généré depuis l'OpenAPI.

## 9.6 Conclusion

L'évaluation confirme la robustesse fonctionnelle et la qualité de la plateforme, ainsi que la valeur de l'intégration Metasploit + RAG. Les limites identifiées ouvrent des pistes d'amélioration claires.

## Conclusion générale

Ce projet de fin d'études a abordé l'automatisation avancée des tests d'intrusion par un système d'IA agentique fondé sur des LLM, en construisant une plateforme complète et cohérente à partir de quatre briques complémentaires. Le moteur agentique Strix fournit l'orchestration multi-agent et l'exécution des outils offensifs dans un bac à sable isolé. Le service Pentest RAG ancre le raisonnement des agents dans un corpus offensif vérifié, réduisant les hallucinations. L'intégration de Metasploit, via l'outil déterministe `exploit_research`, fiabilise la phase la plus délicate — l'exploitation — en déplaçant la décision du modèle vers du code vérifiable et en privilégiant un exécuteur éprouvé, sous le contrôle d'une barrière de validation humaine. Enfin, la console Strix Console transforme cet outillage en un véritable service d'opérations de sécurité, doté d'une authentification forte, d'un contrôle d'accès par rôles, d'un suivi temps réel, de la visualisation de la sortie du moteur, de la post-exploitation, du triage, de la comparaison de scans, de la planification récurrente, du reporting et d'un journal d'audit inaltérable.

Développé avec Next.js et FastAPI, l'ensemble a été validé par 39 tests automatisés et vérifié de bout en bout, jusqu'à la production d'une recommandation d'exploitation ancrée (Drupalgeddon2) sur une cible réelle. Au-delà des résultats obtenus, ce travail confirme la pertinence d'une approche hybride associant l'autonomie des agents IA, la robustesse d'outils éprouvés et la rigueur d'une base de connaissances vérifiée, le tout encadré par une plateforme sûre et gouvernée. Les perspectives — extraction du runner vers une file durable, enrichissement continu du corpus RAG, intégrations métier (Jira, SIEM), cartographie normative des vulnérabilités et génération automatique du client d'API — dessinent une évolution naturelle vers un produit d'entreprise.

## Bibliographie & Webographie

- [1] Strix — Moteur multi-agent open-source de tests d'intrusion. <https://github.com/usestrix/strix>
- [2] Rapid7 — Metasploit Framework. <https://www.metasploit.com>
- [3] Qdrant — Base de données vectorielle. <https://qdrant.tech>
- [4] LangChain — Framework d'orchestration LLM. <https://python.langchain.com>
- [5] Sentence-Transformers / Cross-Encoders. <https://www.sbert.net>
- [6] Nomic Embed — nomic-ai/nomic-embed-text-v1. <https://huggingface.co/nomic-ai>
- [7] OWASP — Web Security Testing Guide et OWASP Top 10. <https://owasp.org>
- [8] PentestGPT — LLM-driven penetration testing. Travaux de recherche associés.
- [9] FastAPI — Framework web asynchrone Python. <https://fastapi.tiangolo.com>
- [10] Next.js — Framework React (App Router). <https://nextjs.org>
- [11] SQLAlchemy 2.0 — ORM Python. <https://www.sqlalchemy.org>
- [12] APScheduler — Ordonnancement de tâches en Python. <https://apscheduler.readthedocs.io>
- [13] MITRE — Common Vulnerabilities and Exposures (CVE). <https://cve.mitre.org>
- [14] Retrieval-Augmented Generation (RAG) — Lewis et al., 2020.